

```
In [144... import pandas as pd
import numpy as np
```

```
In [145... pd.set_option('display.max_rows', None)
pd.set_option('display.max_columns', None)

df = pd.read_csv('cars.csv')
df.head()
```

Out[145...

	Unnamed: 0	symboling	normalized-losses	make	fuel-type	aspiration	num-of-doors	body-style	drive-wheels	engine-location	wheel-base	length	width	height	curb-weight	engine-type
0	0	3	?	alfa-romero	gas	std	two	convertible	rwd	front	88.6	168.8	64.1	48.8	2548	dohc
1	1	1	?	alfa-romero	gas	std	two	hatchback	rwd	front	94.5	171.2	65.5	52.4	2823	ohcv
2	2	2	164	audi	gas	std	four	sedan	fwd	front	99.8	176.6	66.2	54.3	2337	ohc
3	3	2	164	audi	gas	std	four	sedan	4wd	front	99.4	176.6	66.4	54.3	2824	ohc
4	4	2	?	audi	gas	std	two	sedan	fwd	front	99.8	177.3	66.3	53.1	2507	ohc

```
In [146... df.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 204 entries, 0 to 203
Data columns (total 27 columns):
 #   Column                Non-Null Count  Dtype  
---  -
 0   Unnamed: 0            204 non-null   int64  
 1   symboling              204 non-null   int64  
 2   normalized-losses     204 non-null   str     
 3   make                  204 non-null   str     
 4   fuel-type             204 non-null   str     
 5   aspiration            204 non-null   str     
 6   num-of-doors          204 non-null   str     
 7   body-style            204 non-null   str     
 8   drive-wheels          204 non-null   str     
 9   engine-location       204 non-null   str     
10  wheel-base            204 non-null   float64 
11  length                204 non-null   float64 
12  width                 204 non-null   float64 
13  height                204 non-null   float64 
14  curb-weight           204 non-null   int64  
15  engine-type           204 non-null   str     
16  num-of-cylinders      204 non-null   str     
17  engine-size           204 non-null   int64  
18  fuel-system           204 non-null   str     
19  bore                  204 non-null   str     
20  stroke                204 non-null   str     
21  compression-ratio     204 non-null   float64 
22  horsepower            204 non-null   str     
23  peak-rpm              204 non-null   str     
24  city-mpg              204 non-null   int64  
25  highway-mpg           204 non-null   int64  
26  price                 204 non-null   str     
dtypes: float64(5), int64(6), str(16)
memory usage: 43.2 KB

```

In [147... `df.describe()`

Out[147...

	Unnamed: 0	symboling	wheel-base	length	width	height	curb-weight	engine-size	compression-ratio	city-mpg	highway-mpg
count	204.000000	204.000000	204.000000	204.000000	204.000000	204.000000	204.000000	204.000000	204.000000	204.000000	204.000000
mean	101.500000	0.823529	98.806373	174.075000	65.916667	53.749020	2555.602941	126.892157	10.148137	25.240196	30.769608
std	59.033889	1.239035	5.994144	12.362123	2.146716	2.424901	521.960820	41.744569	3.981000	6.551513	6.898337
min	0.000000	-2.000000	86.600000	141.100000	60.300000	47.800000	1488.000000	61.000000	7.000000	13.000000	16.000000
25%	50.750000	0.000000	94.500000	166.300000	64.075000	52.000000	2145.000000	97.000000	8.575000	19.000000	25.000000
50%	101.500000	1.000000	97.000000	173.200000	65.500000	54.100000	2414.000000	119.500000	9.000000	24.000000	30.000000
75%	152.250000	2.000000	102.400000	183.200000	66.900000	55.500000	2939.250000	142.000000	9.400000	30.000000	34.500000
max	203.000000	3.000000	120.900000	208.100000	72.300000	59.800000	4066.000000	326.000000	23.000000	49.000000	54.000000

In [148...

```
df.isnull().sum()
```

```
Out[148... Unnamed: 0      0
            symboling  0
            normalized-losses  0
            make      0
            fuel-type  0
            aspiration  0
            num-of-doors  0
            body-style  0
            drive-wheels  0
            engine-location  0
            wheel-base  0
            length      0
            width      0
            height      0
            curb-weight  0
            engine-type  0
            num-of-cylinders  0
            engine-size  0
            fuel-system  0
            bore        0
            stroke      0
            compression-ratio  0
            horsepower  0
            peak-rpm    0
            city-mpg    0
            highway-mpg  0
            price      0
            dtype: int64
```

```
In [149... (df == '?').sum().nlargest(15)
```

```
Out[149... normalized-losses    40
bore                4
stroke             4
price              4
num-of-doors       2
horsepower         2
peak-rpm           2
Unnamed: 0         0
symboling          0
make               0
fuel-type          0
aspiration         0
body-style         0
drive-wheels       0
engine-location    0
dtype: int64
```

```
In [150... if df['Unnamed: 0'].is_unique:
    df = df.set_index('Unnamed: 0')
    df.index.name = 'car_id'
    print("Index successfully.")
else:
    df = df.drop(columns=['Unnamed: 0'])
    print("Column dropped.")
```

Index successfully.

```
In [151... df = df.replace('?', np.nan)

df['drive-wheels'] = df['drive-wheels'].replace('4wd', 'fwd')
df.head()
```

Out[151...

	symboling	normalized-losses	make	fuel-type	aspiration	num-of-doors	body-style	drive-wheels	engine-location	wheel-base	length	width	height	curb-weight	engine-type	num-of-cylinder
car_id																
0	3	NaN	alfa-romero	gas	std	two	convertible	rwd	front	88.6	168.8	64.1	48.8	2548	dohc	four
1	1	NaN	alfa-romero	gas	std	two	hatchback	rwd	front	94.5	171.2	65.5	52.4	2823	ohcv	six
2	2	164	audi	gas	std	four	sedan	fwd	front	99.8	176.6	66.2	54.3	2337	ohc	four
3	2	164	audi	gas	std	four	sedan	fwd	front	99.4	176.6	66.4	54.3	2824	ohc	five
4	2	NaN	audi	gas	std	two	sedan	fwd	front	99.8	177.3	66.3	53.1	2507	ohc	five

In [152...

```
df = df.astype({
    'normalized-losses': 'Int64',
    'horsepower': 'Int64',
    'peak-rpm': 'Int64',
    'price': 'Int64',
    'bore': 'float',
    'stroke': 'float'
})

df.info()
```

```

<class 'pandas.DataFrame'>
RangeIndex: 204 entries, 0 to 203
Data columns (total 26 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   symboling              204 non-null    int64
 1   normalized-losses      164 non-null    Int64
 2   make                   204 non-null    str
 3   fuel-type              204 non-null    str
 4   aspiration              204 non-null    str
 5   num-of-doors            202 non-null    str
 6   body-style              204 non-null    str
 7   drive-wheels           204 non-null    str
 8   engine-location        204 non-null    str
 9   wheel-base             204 non-null    float64
10  length                 204 non-null    float64
11  width                  204 non-null    float64
12  height                 204 non-null    float64
13  curb-weight            204 non-null    int64
14  engine-type            204 non-null    str
15  num-of-cylinders       204 non-null    str
16  engine-size            204 non-null    int64
17  fuel-system            204 non-null    str
18  bore                   200 non-null    float64
19  stroke                 200 non-null    float64
20  compression-ratio      204 non-null    float64
21  horsepower             202 non-null    Int64
22  peak-rpm               202 non-null    Int64
23  city-mpg               204 non-null    int64
24  highway-mpg            204 non-null    int64
25  price                  200 non-null    Int64
dtypes: Int64(4), float64(7), int64(5), str(10)
memory usage: 42.4 KB

```

```
In [153... df.isnull().sum().nlargest(10)
```

```
Out[153...  normalized-losses    40
           bore         4
           stroke       4
           price        4
           num-of-doors  2
           horsepower    2
           peak-rpm      2
           symboling     0
           make          0
           fuel-type     0
           dtype: int64
```

```
In [154... df.info()
```



```
<class 'pandas.DataFrame'>
RangeIndex: 204 entries, 0 to 203
Data columns (total 26 columns):
#   Column                Non-Null Count  Dtype
---  -
0   symboling              204 non-null    int64
1   normalized-losses      164 non-null    Int64
2   make                   204 non-null    str
3   fuel-type              204 non-null    str
4   aspiration              204 non-null    str
5   num-of-doors           202 non-null    str
6   body-style             204 non-null    str
7   drive-wheels           204 non-null    str
8   engine-location        204 non-null    str
9   wheel-base            204 non-null    float64
10  length                 204 non-null    float64
11  width                  204 non-null    float64
12  height                 204 non-null    float64
13  curb-weight            204 non-null    int64
14  engine-type            204 non-null    str
15  num-of-cylinders       204 non-null    str
16  engine-size            204 non-null    int64
17  fuel-system            204 non-null    str
18  bore                   200 non-null    float64
19  stroke                 200 non-null    float64
20  compression-ratio      204 non-null    float64
21  horsepower             202 non-null    Int64
22  peak-rpm               202 non-null    Int64
23  city-mpg               204 non-null    int64
24  highway-mpg            204 non-null    int64
25  price                  200 non-null    Int64
dtypes: Int64(4), float64(7), int64(5), str(10)
memory usage: 42.4 KB
```

```
In [155... print(df[['normalized-losses', 'bore', 'stroke', 'horsepower', 'peak-rpm', 'price']].skew())
```

```
normalized-losses    0.765976
bore                 0.02708
stroke              -0.695469
horsepower           1.390466
peak-rpm             0.069271
price                1.805801
dtype: Float64
```

```
In [156... def check_outliers(column):
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)
```

```

IQR = Q3 - Q1
lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR
outliers = df[(df[column] < lower) | (df[column] > upper)]
print(f"Column: {column}")
print(f"Number of outliers: {len(outliers)}")
print(f"Lower limit: {lower:.2f}, Upper limit: {upper:.2f}\n")

for col in ['normalized-losses', 'bore', 'stroke', 'horsepower', 'peak-rpm', 'price']:
    check_outliers(col)

```

```

Column: normalized-losses
Number of outliers: 1
Lower limit: 10.00, Upper limit: 234.00

```

```

Column: bore
Number of outliers: 0
Lower limit: 2.49, Upper limit: 4.25

```

```

Column: stroke
Number of outliers: 20
Lower limit: 2.66, Upper limit: 3.86

```

```

Column: horsepower
Number of outliers: 6
Lower limit: 1.00, Upper limit: 185.00

```

```

Column: peak-rpm
Number of outliers: 2
Lower limit: 3750.00, Upper limit: 6550.00

```

```

Column: price
Number of outliers: 14
Lower limit: -5313.62, Upper limit: 29589.38

```

```

In [157... cols_to_fill = ['normalized-losses', 'bore', 'stroke', 'horsepower', 'peak-rpm', 'price']
df[cols_to_fill] = df[cols_to_fill].fillna(df[cols_to_fill].median())

door_mode = df['num-of-doors'].mode()[0]
df['num-of-doors'] = df['num-of-doors'].fillna(door_mode)

```

```

In [158... df.isnull().sum().nlargest(15)

```

```
Out[158... symboling      0
normalized-losses  0
make              0
fuel-type        0
aspiration       0
num-of-doors     0
body-style       0
drive-wheels     0
engine-location  0
wheel-base      0
length          0
width           0
height          0
curb-weight      0
engine-type      0
dtype: int64
```

```
In [159... #-----
#--ANSWER OF THE 5 ASSIGNMENT QUESTIONS--
#-----
```

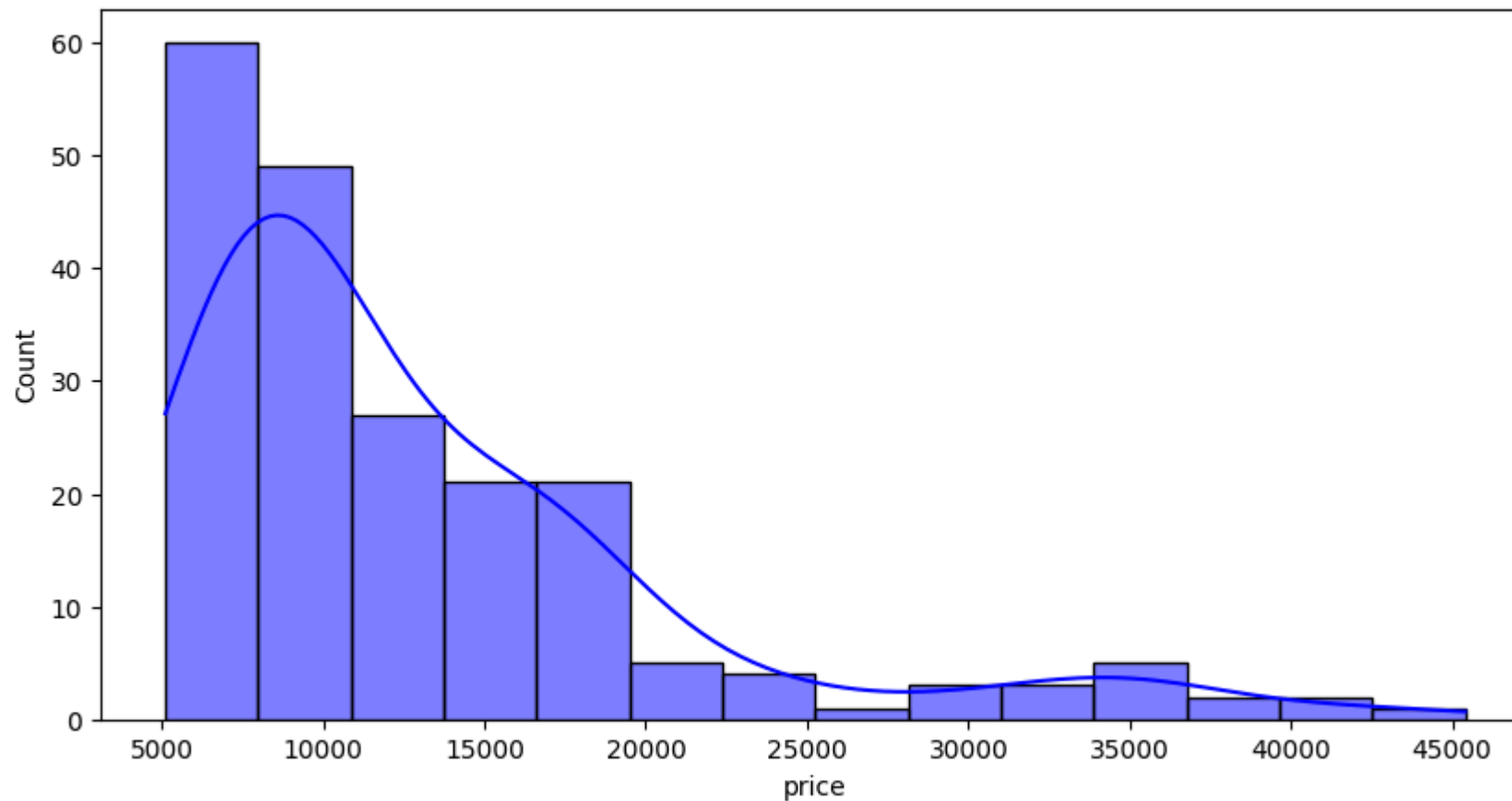
```
In [160... #-Q-1.  How is car price distributed? Is it skewed or normally distributed?
```

```
In [161... import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10,5))
sns.histplot(df['price'], kde=True, color='blue')
plt.title('Car Price Distribution')
plt.show()

print("Skewness:", df['price'].skew())
```

Car Price Distribution



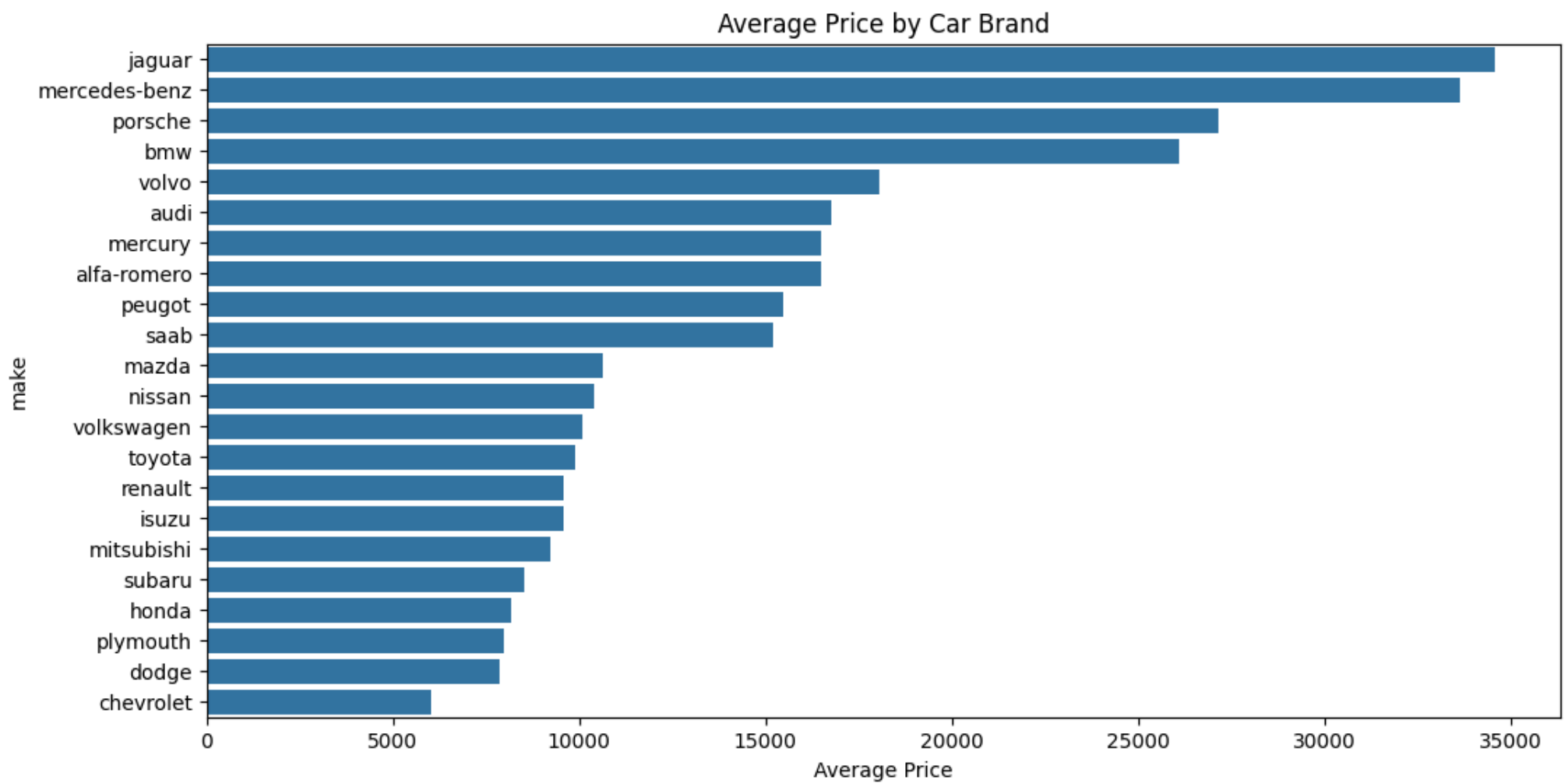
Skewness: 1.8372623688749543

In [162... *--Q-1 Answer: Price is right-skewed. Most cars are cheap, few luxury cars are very expensive.*

In [163... *-Q-2. How does price vary by brand or manufacturer*

```
In [164... plt.figure(figsize=(12,6))
brand_price = df.groupby('make')['price'].mean().sort_values(ascending=False)
sns.barplot(x=brand_price.values, y=brand_price.index)
plt.title('Average Price by Car Brand')
plt.xlabel('Average Price')
plt.show()

brand_price.head(10)
```



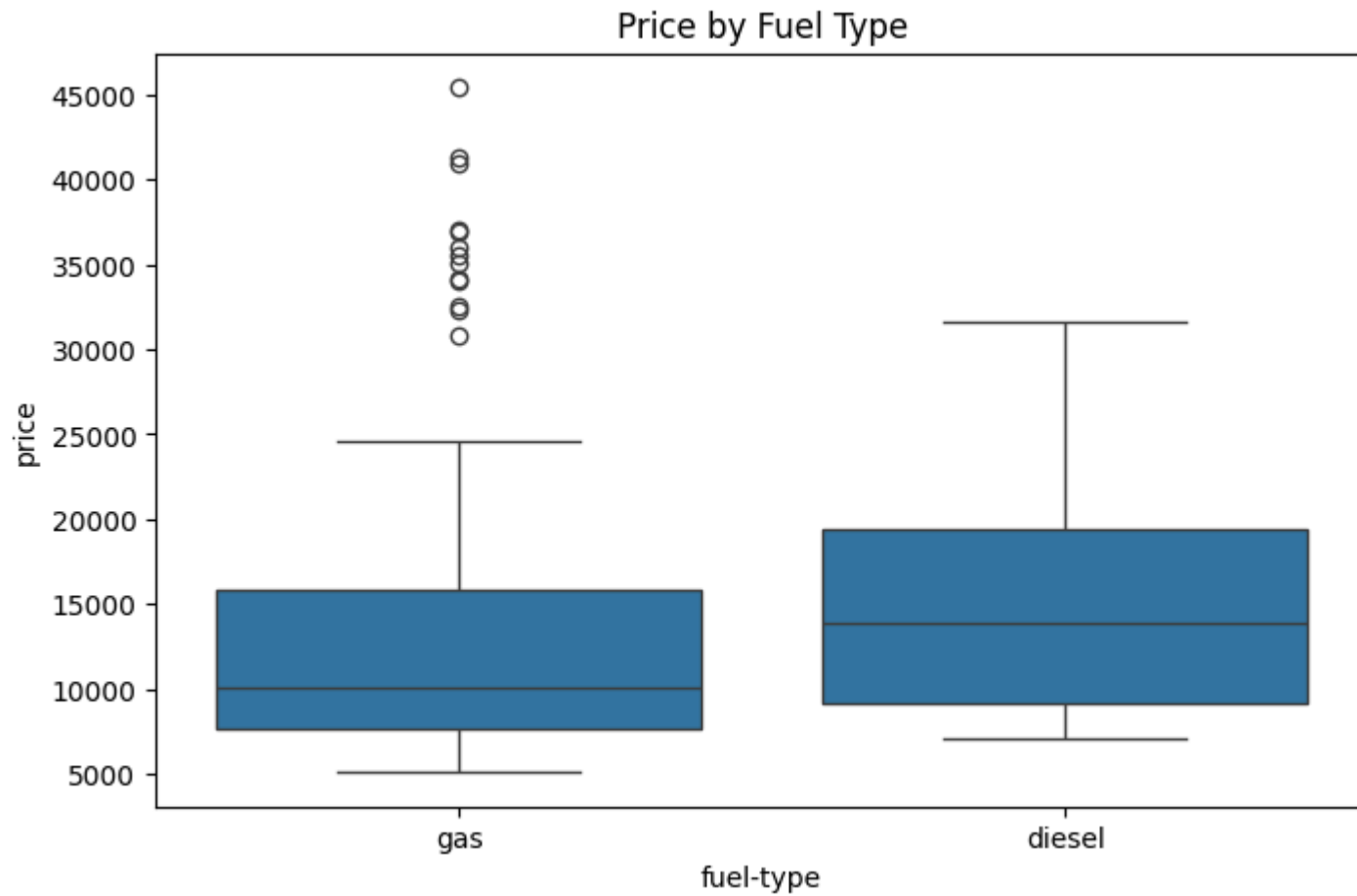
```
Out[164... make
jaguar      34600.0
mercedes-benz 33647.0
porsche     27174.4
bmw         26118.75
volvo       18063.181818
audi        16775.0
mercury     16503.0
alfa-romero 16500.0
peugot      15489.090909
saab        15223.333333
Name: price, dtype: Float64
```

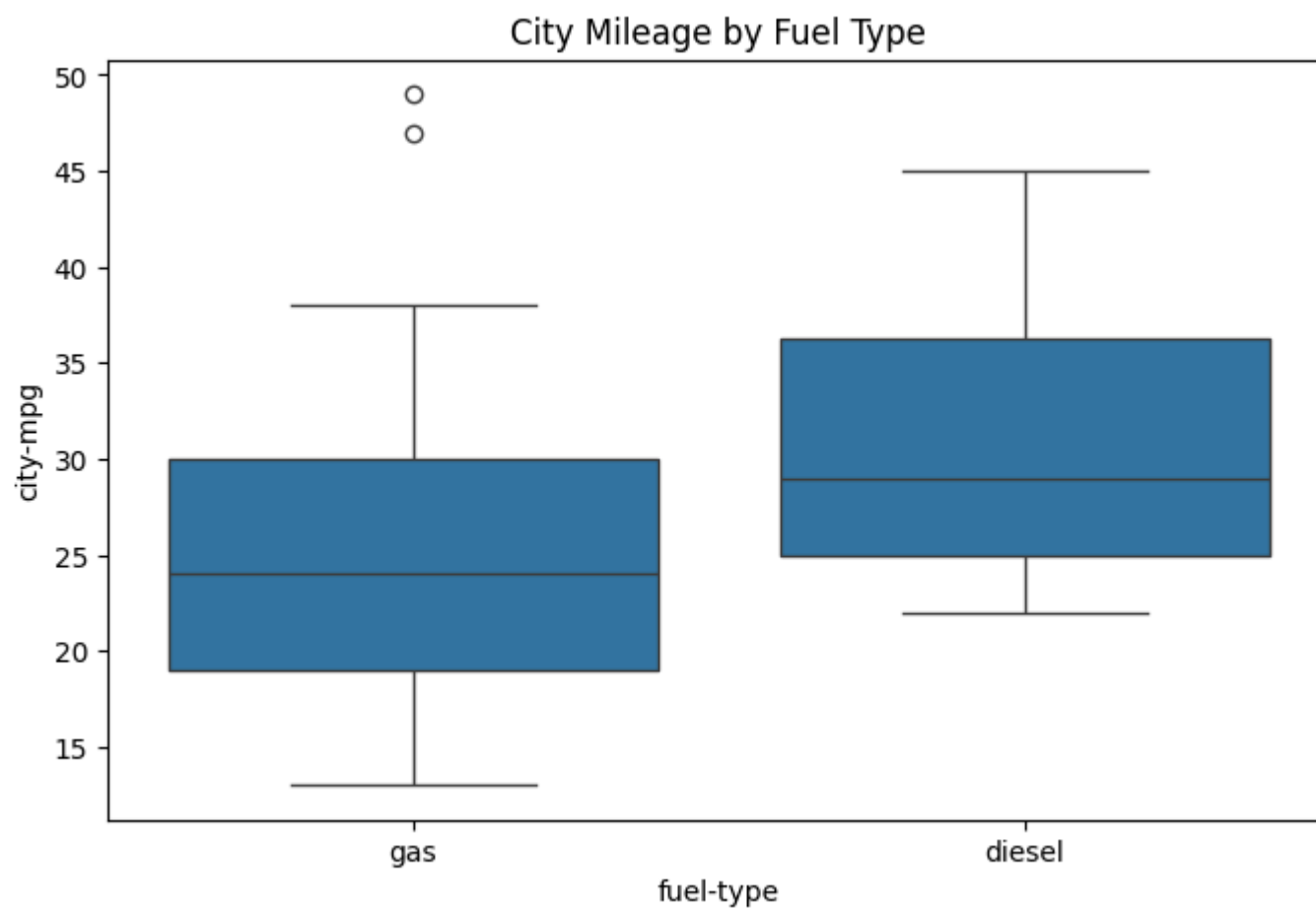
```
In [165... #-Q-3. How does fuel type affect mileage or price?
```

In [166...

```
plt.figure(figsize=(8,5))
sns.boxplot(x='fuel-type', y='price', data=df)
plt.title('Price by Fuel Type')
plt.show()

plt.figure(figsize=(8,5))
sns.boxplot(x='fuel-type', y='city-mpg', data=df)
plt.title('City Mileage by Fuel Type')
plt.show()
```





In [167... *#-Q-4. Which factors most strongly influence car price?*

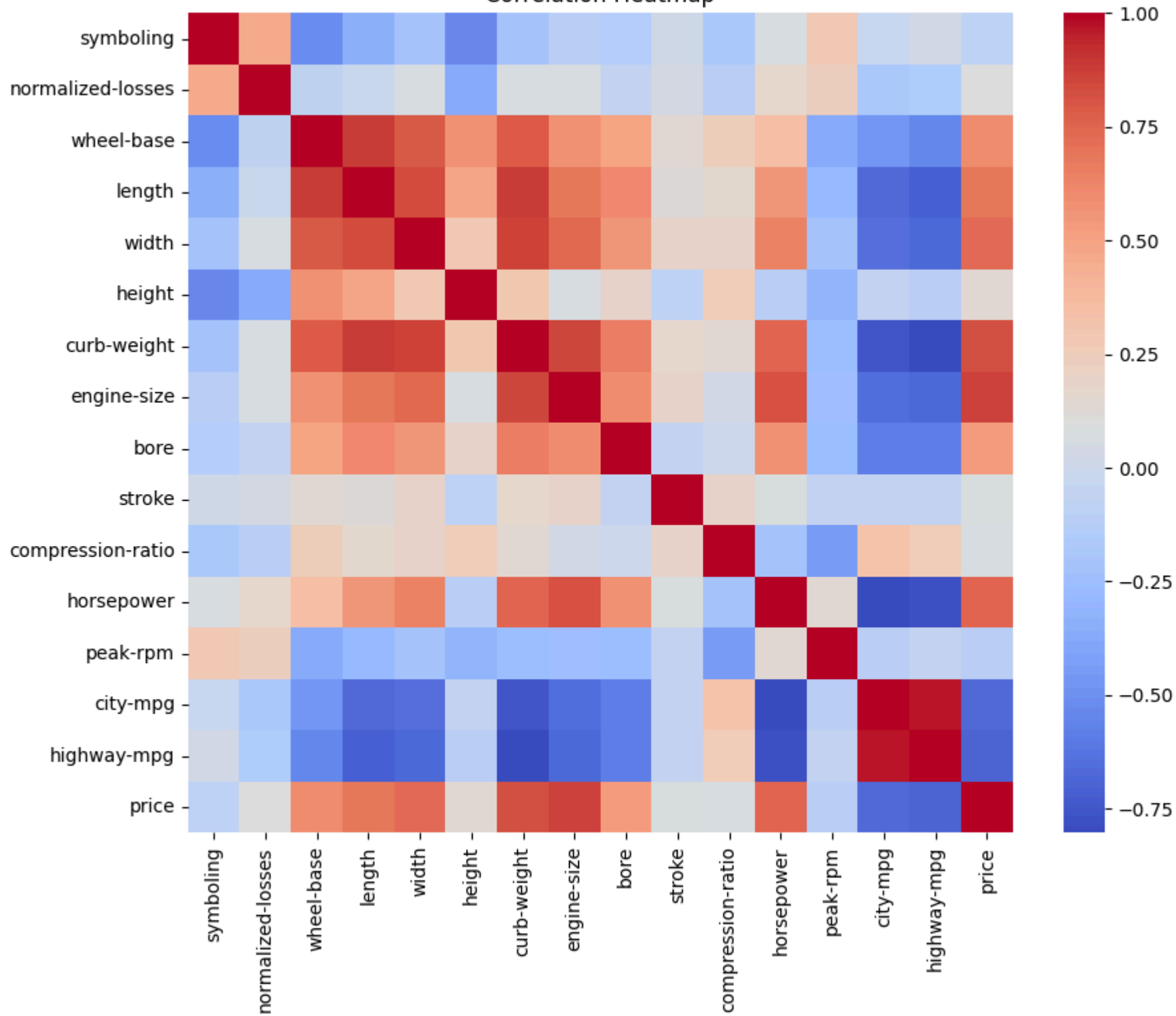
```
In [168... # Correlation with price
corr = df.select_dtypes(include=np.number).corr()['price'].sort_values(ascending=False)
print(corr)

# Heatmap (top 10 features)
plt.figure(figsize=(10,8))
sns.heatmap(df.select_dtypes(include=np.number).corr(), cmap='coolwarm', annot=False)
plt.title('Correlation Heatmap')
plt.show()
```

price	1.000000
engine-size	0.860321
curb-weight	0.819806
horsepower	0.749856
width	0.725971
length	0.686992
wheel-base	0.589365
bore	0.533105
height	0.142356
normalized-losses	0.095543
stroke	0.084744
compression-ratio	0.072983
symboling	-0.081113
peak-rpm	-0.107293
city-mpg	-0.669376
highway-mpg	-0.693444

Name: price, dtype: float64

Correlation Heatmap



```
In [169... --Q-4 Answer: Top factors: engine-size, curb-weight, horsepower, width, Length
```

```
In [170... #-Q-5. What types of cars offer the best value for money (low price, high performance)?
```

```
In [171... df['value_score'] = df['horsepower'] / df['price']

best_value = df.nlargest(10, 'value_score')[['make', 'body-style', 'fuel-type',
                                             'horsepower', 'price', 'value_score']]

best_value
```

```
Out[171...      make  body-style  fuel-type  horsepower  price  value_score
```

car_id						
128	porsche	hatchback	gas	288	10270	0.028043
8	audi	hatchback	gas	160	10270	0.015579
166	toyota	hardtop	gas	116	8449	0.013729
137	subaru	hatchback	gas	69	5118	0.013482
78	mitsubishi	hatchback	gas	102	7689	0.013266
49	mazda	hatchback	gas	68	5195	0.01309
22	dodge	hatchback	gas	102	7957	0.012819
118	plymouth	hatchback	gas	102	7957	0.012819
75	mitsubishi	hatchback	gas	68	5389	0.012618
84	mitsubishi	sedan	gas	88	6989	0.012591

```
In [172... from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression

# Select numeric features only
X = df.select_dtypes(include=['int64', 'float64']).drop('price', axis=1)
y = df['price']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)
```

```
accuracy = model.score(X_test, y_test)
print('-'*62)
print("My Car Price Model Prediction Accuracy is:", accuracy)
print('-'*62)
```

```
-----
My Car Price Model Prediction Accuracy is: 0.8716947277527638
-----
```

In []: